Logistic Regression

Logistic regression predicts the probability that an input belongs to a specific class.

Tell me about this simple machine learning model:

```
X = np.asarray(X); y = np.asarray(y)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=4, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(Xtr, ytr)
pred = model.predict(Xte)
```

This code sets up a standard, production-ready pipeline for Logistic Regression, which is a fundamental machine learning model used for **classification** (predicting categorical labels, like Yes/No, Spam/Not Spam).

Here is a step-by-step breakdown of exactly what each line of this code is doing:

1. Data Preparation

python

```
X = np.asarray(X); y = np.asarray(y)
```

- **What it does:** Converts the features (X) and targets (y) into NumPy arrays.
- **Why it matters:** Ensures the data is in a clean, consistent format that scikit-learn algorithms expect, preventing potential type conflicts if the input was originally a list or a specific pandas DataFrame format.

2. Train-Test Split

python

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=4, stratify=y)
```

- test_size=0.3: Splits the dataset so that **30% of the data is held back for testing** and 70% is used for training.
- random_state=4: Sets a random seed. This ensures that every time you run this code, the data splits exactly the same way, making your results reproducible.
- stratify=y: **Crucial for classification.** This ensures that the proportion of classes (e.g., 80% healthy, 20% sick) stays exactly the same in both the training set and the

testing set. It prevents the model from being evaluated on an unfair or unrepresentative subset of data.

3. Pipeline Construction

python

```
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
```

- StandardScaler(): Scales your features so they have a mean of 0 and a variance of 1. Logistic Regression relies on gradient descent/coordinate descent, which converges much faster and performs better when all features are on the same numeric scale.
- LogisticRegression(max_iter=1000): The actual classification model. The max_iter=1000 increases the maximum number of iterations allowed for the optimization algorithm to find the best fit (the default is often 100, which can cause "failed to converge" warnings on complex datasets).
- make_pipeline(...): Chains these two steps together. This prevents **data leakage** because the scaling transformation will automatically adapt to the training data first, without accidentally "peeking" at the test data.

4. Training and Prediction

python

```
model.fit(Xtr, ytr)
pred = model.predict(Xte)
```

- model.fit(...): Learns the scaling parameters from Xtr and then trains the LogisticRegression model on that scaled data.
- model.predict(...): Automatically scales the unseen test data (Xte) using the exact same scaling parameters learned from the training set, and then outputs the final classification predictions (pred).

Multiclass Logistic Regression Model

Tell me more about this PyTorch model:

```
X = torch.tensor(np.asarray(X), dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)
model = torch.nn.Linear(4, 3)
opt = torch.optim.Adam(model.parameters(), lr=0.04)
for _ in range(120):
    opt.zero_grad()
    loss = torch.nn.functional.cross_entropy(model(X), y)
    loss.backward(); opt.step()
    acc = (model(X).argmax(1) == y).float().mean().item()
```

This code defines and trains a multiclass logistic regression model (a single-layer neural network) using PyTorch, most likely operating on the classic Iris dataset.

Here is a detailed breakdown of what this exact script is doing:

📋 Model Architecture & Data

- **The Task:** It is solving a **3-class classification problem** using **4 input features**. This exact shape (4 features, 3 classes) is the hallmark of the Iris flower dataset (predicting 3 species based on sepal/petal length and width).
- **The Network:** torch.nn.Linear(4, 3) creates a single linear layer with 4 input nodes and 3 output nodes. It computes \(Y = XW^T + b\).
- **Data Types:**
    - Features (X) are converted to float32 (standard for neural network weights).
    - Labels (y) are converted to long (64-bit integers), which PyTorch requires for classification targets.

⚙️ Training Setup

- **Optimizer:** It uses the **Adam optimizer** with a relatively high learning rate (lr=0.04). Adam dynamically adjusts learning rates per parameter, helping the model converge quickly.
- **Loss Function:** cross_entropy automatically applies a Softmax function to the 3 raw network outputs (logits) to turn them into probabilities, then computes the negative log-likelihood loss against the true labels y.

🔄 The Optimization Loop (120 Epochs)

For 120 iterations, the code performs full-batch gradient descent:

1. opt.zero_grad() clears out the gradients from the previous step so they don't accumulate.
2. model(X) runs the forward pass to get predictions.
3. loss.backward() calculates the gradients of the loss with respect to the weights and biases using backpropagation.
4. opt.step() updates the model's weights to minimize the loss.

📊 Performance Metric

The last line calculates the final **training accuracy**:

- model(X).argmax(1) finds the index (0, 1, or 2) of the highest predicted value for each sample.
- == y compares those predictions to the actual targets, resulting in a boolean tensor (True/False).
- .float().mean().item() converts those booleans to 1.0 or 0.0, averages them to get the accuracy percentage, and extracts it as a standard Python number.

Would you like to know how to **split this into train/test sets** to evaluate it properly, or would you like to see how to **save and load** this trained model?